

LTX-Video 2 / LTXAV

Character Consistency at Inference Time — Research Findings

FunPack Project · May 2026 · Hardware: NVIDIA RTX PRO 6000

This document summarises all techniques we explored for maintaining character appearance consistency across multi-scene video generations using LTX-Video 2 (22B, LTXAV variant) within the FunPack ComfyUI node pack. It covers what works, what doesn't, why, and the empirical data collected along the way. It is intended as a reference for any developer attempting similar inference-time interventions on LTX2.

1. Model Architecture

LTX-Video 2 (LTXAV) is a 48-block dual-stream transformer generating synchronised audio and video in a single forward pass. The video stream has 14B parameters; the audio stream has 5B. ComfyUI loads it as **ModelPatcherDynamic** for dynamic VRAM management.

Blocks	Zone	Role
0–13	Early	Texture, low-level noise, grain
14, 19	Semantic anchors	Scene composition, structure (PAG/STG defaults)
20–35	Concept / identity	Character appearance, proportions, object identity
36–47	Late refinement	High-level detail, sharpness

Distilled sigma schedule (Lightricks recommended, 8 steps):

[1.0, 0.99375, 0.9875, 0.98125, 0.975, 0.909375, 0.725, 0.421875, 0.0]

Steps 1–5 (σ 1.0→0.975) are a near-pure noise plateau with almost no signal. Meaningful structure emerges only at step 6 (σ =0.909). Character form is set at step 7 (σ =0.725); fine details at step 8 (σ =0.421). Any technique that depends on sigma thresholds must be calibrated against the actual schedule in use — general assumptions about sigma distributions fail for distilled models.

2. Inference-Time Mechanisms — What We Tried

2.1 patches_replace["dit"]

ComfyUI provides a patches_replace["dit"] dict in transformer_options that is intended to let external code replace individual transformer block forward functions. We populated entries for blocks 14, 19, 21, 26, 31 with capture/inject functions.

Diagnostic: transformer_options confirmed present in args["c"] with our 5 entries (to_patches=5, model_to_patches=5). However, capture_buf remained empty across all 8 denoising steps — the blocks never called our replacement functions.

Conclusion: LTXAV's transformer blocks do not check for patches_replace["dit"]. The dict reaches transformer_options but is silently ignored. This mechanism is effectively a no-op for LTXAV.

2.2 model.apply_model()

We attempted to run a single reference forward pass at $\sigma \approx 0.03$ to capture hidden states before the main generation begins.

Result: AttributeError — ModelPatcherDynamic has no apply_model method. The standard ModelPatcher API does not apply here.

2.3 Separate reference forward pass via apply_fn

We passed apply_fn (available inside model_function_wrapper) with a custom reference tensor and $\sigma = 0.03$. This avoided the apply_model issue but caused a shape mismatch: the conditioning tensors in args["c"] were prepared for the full generation input shape, not our custom reference tensor.

Result: RuntimeError — tensor reshape failed. Conditioning shapes are tightly coupled to the input latent dimensions.

2.4 PyTorch forward hooks — WORKING

We registered nn.Module forward hooks directly on the transformer block objects:

```
model.model.diffusion_model.transformer_blocks[idx].register_forward_hook(fn)
```

Discovery path: iterate ["transformer_blocks", "blocks", "joint_blocks", "layers"] on diffusion_model; use first ModuleList with length ≥ 28 . Block output format for LTXAV: dict with key "img" — out["img"] is the hidden state tensor.

- Hooks fire on every block call regardless of patches_replace support
- Hooks CAN modify the output by returning a non-None value
- Hooks registered lazily (only when $\sigma < 0.95$) eliminate overhead on steps 1–5
- Tensors kept on GPU throughout — CPU↔GPU round trips caused 3–4× slowdown

Performance note: 150-frame 22B LTXAV, RTX PRO 6000 — lazy hook registration reduced overhead from ~40s to ~12s per generation.

3. Hidden State Injection — Lessons Learned

Hidden states at all transformer blocks encode the **full scene context** — composition, pose, camera angle, background, lighting — not just character identity. There is no clean separation between "who this character is" and "how this scene is composed" at the block representation level.

3.1 Injection strength calibration

Strength	Effect
22%	Full scene bleeding — reference scene reproduced almost exactly
10%	Still too strong — composition from reference overrides new scene
4%	Subtle character bias without obvious scene bleed — marginal
2%	Negligible — character looks unrelated to reference

The usable window is extremely narrow. The entanglement of scene context and character identity in the hidden state representation means any meaningful injection will also bias scene composition.

3.2 Block similarity — empirical data

We measured cross-scene cosine similarity of hidden states at $\sigma=0.909$ across 5 scenes of the same character in different compositions (using forward hooks for capture):

Block	Similarity	Block	Similarity	Block	Similarity
b14	0.609	b22	0.507	b29	0.480
b33	0.581	b31	0.506	b28	0.478
b34	0.551	b35	0.509	b23	0.474
b30	0.542	b32	0.534	b24	0.457
b21	0.538	b20	0.532	b25	0.438
b19	0.522	b29	0.480	b27	0.437
				b26	0.403

No block exceeds 0.67 similarity. There is no clean identity/scene separation. The highest blocks (b14, b33, b20, b21, b30) became IDENTITY_BLOCKS = [14, 20, 21, 30, 33]. Blocks b26 and b27 (most scene-specific) were removed.

With carry_i2v_guides=True: b14:0.785, b33:0.742, b20:0.739 — all scenes started from the same reference frame, biasing similarity upward. These numbers are not clean.

4. Bridge Shots — Attempted, Abandoned

Inspired by LTX Desktop's Bridge Shots feature (first-last-frame conditioned generation), we attempted to generate smooth transition clips between scenes.

Attempt 1: Bridge to reference image

After generating scene N, bridge to the i2v reference image as the target, then start scene N+1 from the reference. **Result:** visible reset — the character visually snaps back to the static reference pose between every scene.

Attempt 2: Two-pass — bridge between actual scene frames

Generate all scenes independently first, then generate bridges using scene N's last frame and scene N+1's first frame as dual anchors (first-last-frame conditioning). **Result:** identical reset artifact. Root cause: every independently generated scene starts from the i2v reference frame (the anchor). Scene N+1's first frame IS the reference image. The bridge always targets the reference.

Root cause: LTX-2 anchors extremely strongly to pinned frames in first-last-frame mode. Without a mechanism to generate scene N+1 with a non-reference first frame while maintaining character consistency (i.e., IP-Adapter), there is no clean scene to bridge to. Feature removed.

5. What Actually Works

Character consistency in LTX-Video 2 multi-scene generation, in descending order of effectiveness:

5.1 carry_i2v_guides (primary mechanism)

Carries the protected frames from the i2v latent template into the start of each continuation chunk. This explicitly re-presents the reference image to the model at every scene boundary. Result: when scene composition stays close to the reference, character details are reproduced faithfully with no prompt engineering required.

- Works well: same or similar composition to reference
- Limitation: anatomy artifacts when reference pose conflicts with scene action — LoRAs help
- Limitation: dramatic composition changes (different camera angle) break fine details like ear shape

5.2 Reference injection via forward hooks (supplementary)

Captures hidden states from IDENTITY_BLOCKS at $\sigma \approx 0.91$ during scene 1, then injects them at 2% strength during steps 7–8 (σ gate 0.35–0.80) of subsequent scenes. Adds ~12s overhead per generation on RTX PRO 6000. Opt-in via Studio 'Reference injection' toggle (default off). Only meaningful alongside carry_i2v_guides.

- Reduces inter-scene drift marginally for consistent compositions
- Cannot substitute for carry_i2v_guides — proven by testing without it
- Overhead: ~12s per 150-frame 22B generation due to lazy hook registration

5.3 What this combination cannot solve

- Hard composition changes (different camera angle, different pose) — character identity extrapolation from a single angle requires IP-Adapter or trained multi-view data
- Scene transitions — Bridge Shots failed; first-last-frame anchoring always resolves to the reference image
- Sub-2% injection strength — below the noise floor, no measurable effect

6. Conclusions

LTX-Video 2's transformer hidden states are deeply entangled — character identity and scene composition cannot be separated at the block level with inference-time techniques alone. The maximum achievable consistency via hidden state injection is a subtle bias, not reliable identity preservation.

The primary consistency mechanism for LTX-Video 2 is **carry_i2v_guides**: explicit frame anchoring at scene boundaries. This works well when scene compositions stay close to the reference. For cross-angle consistency, the model needs to have been trained with multi-view reference conditioning — which LTX-Video 2's base model was not.

The correct tool for robust cross-angle character consistency is **IC-LoRA** (Identity-Consistent LoRA, available for LTX-2.3) or an IP-Adapter equivalent. These operate by conditioning the entire generation on reference image tokens, which is qualitatively different from post-hoc hidden state injection. Lightricks'

own IC-LoRA Union-Control and Motion-Track-Control models are the recommended path for production character consistency workflows.

FunPack's hidden state injection infrastructure (forward hooks, lazy registration, sigma-gated strength, IDENTITY_BLOCKS empirically determined) is the correct scaffolding to extend with IC-LoRA or similar when such integrations become available.

7. Technical Reference

Model function wrapper args (LTXAV)

```
args.keys() = ['input', 'timestep', 'c', 'cond_or_uncond']
args['c'].keys() = ['c_crossattn', 'frame_rate', 'denoise_mask',
'audio_denoise_mask', 'latent_shapes', 'transformer_options']
```

i2v latent format

```
samples: [1, 128, frames, H, W] – video only (not NestedTensor for i2v template)
noise_mask: [1, 1, frames, 1, 1] – mask = 1 - strength for image frames, 1.0 for noise
Detection: mask[:, :, 0].float().mean() < 0.95 (not < 0.1 as commonly assumed)
```

Forward hook path

```
model.model.diffusion_model.transformer_blocks[idx] # ModuleList, len=48
Block output: dict with 'img' key – out['img'] is the hidden state tensor
Return non-None from hook to modify block output
```

Working configuration

Parameter	Value	Notes
IDENTITY_BLOCKS	[14, 20, 21, 30, 33]	Empirically determined from cosine similarity
Inject strength	2%	Higher causes scene bleed
Inject σ gate	(0.35, 0.80)	Steps 7–8 of 8-step schedule
Snapshot threshold	$\sigma < 0.95$	After step 5, $\sigma=0.909$
Hook registration	Lazy ($\sigma < 0.95$)	Steps 1–5 have zero hook overhead
Overhead	~12s / 150-frame 22B	RTX PRO 6000

8. Scene Chain Sampler — Latent Blending and Transitions

This section documents implementation findings from the FunPackLTXAVSceneChainSampler — the multi-scene continuation node that stitches independently generated scenes into a single video. Most of the issues described below are consequences of the LTXAV VAE's non-linearity and the mismatch

between latent-space operations and their pixel-space results.

8.1 Latent blending at scene boundaries

Scene continuations are joined by blending the overlap region in latent space: $\alpha * \text{scene_N_tail} + (1-\alpha) * \text{scene_}\{N+1\}_\text{start}$, where α is a linear ramp across `video_overlap` latent frames. This blend is non-linear in pixel space — the VAE decode of an interpolated latent is not a linear interpolation of the two decoded images. The result is a zone of `video_overlap` latent frames (~120 pixel frames with typical settings) at each scene boundary that decodes to unexpected or color-shifted content.

Initially diagnosed as a sigma schedule problem. Later attributed to NAG (Negative Attention Guidance) connected during sampling — NAG causes a latent channel bias that decodes as a yellow cast. Both effects can coexist; NAG is the dominant cause when present. Confirmed by disconnecting NAG and observing clean output with unchanged schedule.

8.2 Transition effects — coverage requirement

Visual transition effects (`fade_to_black`, `blur_out_in`, `crossfade`) are applied in pixel space after a single VAE decode of the full video. The initial implementation applied the effect over `transition_duration // 2` pixel frames on each side of the boundary. This was far smaller than the blended-latent zone:

```
blend zone half-width = (video_overlap // 2) * time_scale = 8 * 8 = 64 px  
old fade half = transition_duration // 2 = 8 px (at default 16) blended frames NOT hidden = 64 - 8 = 56 px each side
```

The 56 uncovered frames decoded with color artifacts (yellow or other) and were fully visible in the output, making transitions look like generated artefacts rather than applied post-processing. Fix: the effect half-width is now `blend_half_pixels + transition_duration // 2`, guaranteeing full coverage of the blended zone regardless of `transition_duration`.

8.3 Effect curve shape — flat zone over the blend region

Extending the half-width to cover the blend zone introduced a secondary problem. The original linear V-curve ($\text{brightness} = |i - \text{center}| / \text{half}$) leaves content at the blend zone boundary (~89% brightness at the outer edge) still mostly visible. A wider linear fade does not hide the blended-latent content — it just makes it appear more gradually.

Correct approach: the brightness (or blur sigma) must be zero/maximum flat across the entire blend zone, then ramp back to full only over the extra `transition_duration // 2` frames beyond the zone into clean scene content:

```
brightness = max(0, (|i - center| - blend_half) / extra_half)  
sigma = 8.0 * (1 - max(0, (|i - center| - blend_half) / extra_half))
```

This ensures all blended-latent frames are fully suppressed (black / maximum blur) and only clean scene content on either side fades in/out smoothly. The blend zone content is never visible regardless of the `transition_duration` setting.

8.4 Blur edge artifact — reflect padding

The `blur_out_in` effect uses a box blur (`avg_pool2d`) with kernel size up to 25x25 at maximum sigma. PyTorch's `avg_pool2d` pads with zeros by default, which darkens pixels near image edges when the kernel overlaps the boundary — visually: dark halos on all four sides of blurred frames.

Fix: pre-pad the frame tensor in reflect mode (`torch.nn.functional.pad` with `mode='reflect'`), then run `avg_pool2d` with `padding=0`. Reflected content is averaged instead of zeros, preserving edge brightness.

8.5 Transition disable — minimum duration

The original node enforced `min=2` for `transition_duration`. This meant visual effects could never be fully disabled from the node UI — only setting `effect='none'` on all scenes bypassed them. Additionally, the latent re-encode path enforced `max(2, ...)` for `half_lat` regardless of the user-specified duration.

Fix: `min=0` for `transition_duration`. When `transition_duration=0`, no boundary entries are recorded and all transition code paths are bypassed entirely.

8.6 Memory leak — text encoder held by loop variables

ComfyUI's model management reports a memory leak ("LTXAVTEModel referenced from somewhere") when the text encoder cannot be unloaded after sampling completes. The initial fix deleted only the top-level model, positive, and negative parameters. This was insufficient because the sampling loop leaves several additional variables alive that hold conditioning references:

Variable	Why it keeps the TE alive
<code>scene_conditionings</code>	Slice of positive — holds the original conditioning list entries
<code>scene_positive</code>	Last scene's modified conditioning with <code>guide_attention_entries</code>
<code>scene_negative</code>	Reference to negative (del negative removes name, not object)
<code>chunk</code>	Last continuation chunk — conditioning injected into it
<code>sampled</code>	Last sampled output — may hold references through guide conditioning

Fix: delete all six variables together after the loop and before VAE decode. Python's `del` removes the name binding; the object is freed only when reference count reaches zero. Deleting only some of the references is not sufficient.

8.7 frame_overlap=0 — diagnostic note

Setting `frame_overlap=0` completely disables scene continuation. With `overlap=0`, `_build_continuation_chunk` produces a mask of all-ones (full noise) — equivalent to a fresh i2v generation starting from the template each time. Symptoms:

- Every scene 'restarts from the reference image' — the i2v template anchor is the first frame
- Inference time is identical for all scenes — no overlap means no pinned frames, full denoising budget on every scene
- No visual continuity between scenes regardless of `carry_i2v_guides` state

This is expected behavior, not a bug. Documented here because the symptoms (apparent scene reset, identical timing) are easily mistaken for a node malfunction.